

# Modelado de Base de Datos — Rompecabezas en Neo4j

CC3089 Base de Datos 2 | Universidad del Valle de Guatemala | Semestre I 2026

- Juan Menéndez Guerra
- José Alejandro Antón Escobar
- Jose Pablo Ordoñez Barrios
- Gadiel Ocaña

## 1. Introducción

Este documento describe el modelo de datos, las decisiones de diseño y el algoritmo de ensamblaje desarrollados para el Proyecto 03 de CC3089 Base de Datos 2. El objetivo es modelar un rompecabezas físico en una base de datos de grafos (Neo4j Aura) y proveer un algoritmo que, partiendo de cualquier pieza, indique los pasos para armar el rompecabezas completo o parcialmente cuando hay piezas faltantes.

## 2. Justificación de la tecnología elegida

El problema del rompecabezas es, por naturaleza, un problema de grafo: cada pieza es un nodo y la adyacencia física entre piezas es una arista. Evaluar esta estructura en distintas tecnologías permite justificar la elección de Neo4j.

| Criterio               | PostgreSQL (SQL)                                                   | MongoDB (NoSQL doc.)                                           | Neo4j (Grafos) ✓                                                      |
|------------------------|--------------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------------------|
| Modelado de adyacencia | Tabla puente pieza_conexion con FK; requiere JOIN para cada vecino | Arrays de IDs referenciados; \$lookup para resolver            | Relaciones nativas :CONECTA entre nodos; sin JOIN                     |
| Consulta de vecinos    | SELECT con JOIN anidado; O(n) por nivel de profundidad             | \$lookup múltiple; costoso para grafos profundos               | MATCH BFS nativo; O(V+E) total para todo el rompecabezas              |
| Escalabilidad          | Tablas crecen linealmente; índices críticos para n > 10 000        | Documentos grandes; rendimiento baja con referencias profundas | Grafo escala con sharding; rendimiento estable para millones de nodos |
| Flexibilidad de schema | Schema rígido; cada nuevo atributo                                 | Flexible; cada documento puede                                 | Flexible; propiedades opcionales por nodo                             |

|                      |                                                  |                                                        |                                                                         |
|----------------------|--------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------|
|                      | requiere ALTER TABLE                             | tener campos distintos                                 | sin migración                                                           |
| Piezas faltantes     | Columna booleana activa; filtro en WHERE         | Campo booleano activa; filtro en query                 | Propiedad activa; excluida automáticamente en BFS                       |
| Lenguaje de consulta | SQL estándar; verboso para traversal de grafos   | MQL; expresivo para documentos pero no para relaciones | Cypher; diseñado para patrones de grafo; más legible para este problema |
| Hosting / demo       | Requiere servidor o RDS; configuración adicional | MongoDB Atlas (gratuito); sencillo                     | Neo4j Aura (gratuito); acceso inmediato sin servidor propio             |

**Conclusión:** Neo4j Aura es la tecnología óptima para este problema. La adyacencia entre piezas se modela directamente como aristas, el algoritmo BFS es nativo en Cypher, y el manejo de piezas faltantes con la propiedad activa integra de forma natural con las consultas de traversal.

### 3. Decisiones de diseño

#### 3.1 Nodo raíz :Puzzle en lugar de auto-referencias recursivas

Se usa un nodo raíz (:Puzzle) como punto de entrada al grafo. La alternativa —auto-referencias recursivas entre piezas— dificulta la identificación del punto de inicio y complica las consultas de metadatos del rompecabezas.

| Criterio           | Auto-referencia recursiva                | Nodo raíz :Puzzle (elegido)           |
|--------------------|------------------------------------------|---------------------------------------|
| Claridad semántica | El grafo no tiene raíz definida          | Punto de entrada explícito            |
| Consulta de inicio | MATCH (p:Pieza) WHERE p.es_inicio = true | MATCH (pz:Puzzle)-[:PRIMER_PASO]->(p) |
| Extensibilidad     | Metadatos del puzzle dispersos en piezas | Metadatos centralizados en un nodo    |
| Validación         | Difícil evitar ciclos no intencionados   | Estructura limpia sin ciclos          |

#### 3.2 Propiedad “activa” en lugar de eliminar nodos

Las piezas faltantes no se eliminan del grafo. En cambio, se marca la propiedad activa = false. Esto preserva la estructura de adyacencia completa del rompecabezas, permite indicar los espacios vacíos durante el armado, y hace que la operación sea reversible (la pieza puede recuperarse sin recrear relaciones).

### 3.3 Label único :Pieza sin subtipos

Nuestro modelo original contemplaba labels combinados

(:Pieza:Esquina, :Pieza:Borde, :Pieza:Interior). Para rompecabezas asimétricos y físicos, esta clasificación no es práctica: las piezas no tienen bordes uniformes que permitan distinguir automáticamente esquinas de bordes. La clasificación manual resultaría arbitraria y propensa a errores.

En su lugar, se usa un único label :Pieza para todas las piezas y el usuario designa explícitamente la pieza de inicio al cargar el rompecabezas. Esto simplifica el modelo, elimina ambigüedad y funciona correctamente para cualquier forma de rompecabezas.

## 4. Modelo de datos

### 4.1 Nodos

#### 4.1.1 :Puzzle

Nodo raíz único por rompecabezas. Sirve como punto de entrada al grafo y contiene los metadatos generales.

| Propiedad    | Tipo    | Descripción                                            |
|--------------|---------|--------------------------------------------------------|
| puzzle_id    | String  | Identificador único autogenerado (ej. puzzle_a1b2c3d4) |
| nombre       | String  | Nombre descriptivo del rompecabezas                    |
| total_piezas | Integer | Número total de piezas                                 |
| dimensiones  | String  | Descripción del formato (ej. '500 piezas asimétrico')  |
| imagen_url   | String  | URL de la imagen de referencia (opcional)              |

#### 4.1.2 :Pieza

Label base para todas las piezas del rompecabezas. No se usan subtipos: cualquier pieza puede ser punto de inicio.

| Propiedad   | Tipo    | Descripción                                             |
|-------------|---------|---------------------------------------------------------|
| pieza_id    | String  | Identificador único (P_1, P_2, ..., P_N)                |
| descripcion | String  | Descripción opcional de la pieza (forma, color, etc.)   |
| activa      | Boolean | true = disponible para armar;<br>false = pieza faltante |

## 4.2 Relaciones

| Tipo        | Origen → Destino | Propiedades                                | Descripción                                                                 |
|-------------|------------------|--------------------------------------------|-----------------------------------------------------------------------------|
| PRIMER_PASO | :Puzzle → :Pieza | —                                          | Indica la pieza desde la cual iniciar el armado (designada al cargar)       |
| CONECTA     | :Pieza → :Pieza  | direccion,<br>compatible,<br>fuerza_encaje | Adyacencia física entre piezas vecinas. Se crea bidireccional (dos aristas) |

### Propiedades de CONECTA

| Propiedad     | Tipo    | Valores posibles           | Descripción                                 |
|---------------|---------|----------------------------|---------------------------------------------|
| direccion     | String  | NORTE / SUR / ESTE / OESTE | Dirección de P_origen hacia P_destino       |
| compatible    | Boolean | true                       | Indica que las piezas encajan correctamente |
| fuerza_encaje | Float   | 1.0                        | Certeza del encaje (1.0 = certero)          |

Nota sobre bidireccionalidad: Neo4j almacena relaciones dirigidas. Para representar adyacencia bidireccional se crean dos aristas explícitas: P\_A -[CONECTA {direccion:'ESTE'}]-> P\_B y P\_B -[CONECTA {direccion:'OESTE'}]-> P\_A.

## 4.3 Diagrama del grafo

Representación simplificada de la estructura:

```
(:Puzzle {puzzle_id:'puzzle_abc', nombre:'Mi Rompecabezas',
total_piezas:9})
|
| :PRIMER_PASO
v
(:Pieza {pieza_id:'P_1', activa:true})
|                                     ^
| :CONECTA {direccion:'ESTE'}      | :CONECTA {direccion:'OESTE'}
v                                     |
(:Pieza {pieza_id:'P_2', activa:true})
|
| :CONECTA {direccion:'SUR'}
v
(:Pieza {pieza_id:'P_5', activa:false}) ← pieza faltante
```

## 5. Algoritmo de ensamblaje

### 5.1 Descripción general (BFS)

El algoritmo implementa una búsqueda en anchura (BFS — Breadth-First Search) sobre el grafo de piezas. Partiendo de una pieza inicial, visita todas las piezas conectadas en orden de cercanía, imprimiendo en cada paso qué pieza colocar y con cuáles piezas vecinas se ensambla.

| Elemento             | Descripción                                                                             |
|----------------------|-----------------------------------------------------------------------------------------|
| Entrada              | puzzle_id (requerido), pieza_inicio (opcional; default = PRIMER_PASO)                   |
| Estructura           | Cola BFS (deque), conjunto visitados, conjunto faltantes_vistos                         |
| Por cada paso        | Pieza actual → consulta vecinos<br>via :CONECTA → clasifica cada vecino                 |
| Clasificación vecino | activa=false → [ESPACIO VACÍO]   no visitado<br>→ Ensamblar   ya visitado → ya colocada |
| Salida               | Listado paso a paso + resumen final<br>(ensambladas / total, estado COMPLETO o PARCIAL) |
| Complejidad          | $O(V + E)$ donde V = piezas, E = conexiones<br>CONECTA                                  |

### 5.2 Manejo de piezas faltantes

Antes de ejecutar el BFS, el usuario puede declarar piezas faltantes. El sistema actualiza activa = false en Neo4j (cambio persistente). Durante el BFS, las piezas con activa = false no se agregan a la cola; en su lugar se imprime un mensaje indicando que se debe dejar el espacio vacío. La misma pieza faltante no se reporta más de una vez aunque sea vecina de múltiples piezas.

### 5.3 Formato de salida por paso

```
Paso 001 | Pieza actual: P_1
  → ESTE : Ensamblar 'P_2'
  → NORTE: 'P_4' ya colocada
  → SUR  : [ESPACIO VACÍO] – pieza 'P_3' faltante, dejar el espacio libre
```

```
-----
Paso 002 | Pieza actual: P_2
  → OESTE: 'P_1' ya colocada
  → SUR  : Ensamblar 'P_5'
```

```
-----
...
```

```
Resultado:
Piezas ensambladas : 8 / 9
Piezas faltantes   : 1 → P_3
Estado             : Rompecabezas PARCIALMENTE armado
```

## 5.4 Escalabilidad

El algoritmo BFS tiene complejidad  $O(V + E)$ , donde  $V$  es el número de piezas y  $E$  el número de conexiones. Para un rompecabezas de 500 piezas con conectividad promedio de 3 aristas por pieza, el grafo tiene ~1 500 aristas. Neo4j utiliza un índice en la propiedad activa que acelera el filtrado de piezas faltantes. La solución escala a rompecabezas de miles de piezas sin cambios en el código.

## 6. Implementación del modelo

### 6.1 Script de carga: cargar\_rompecabezas.py

Aplicación interactiva de terminal que carga un rompecabezas en Neo4j Aura.

| Paso | Pregunta / acción                                                                                                                               |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Confirmación destructiva: se eliminarán todos los nodos existentes                                                                              |
| 2    | Metadatos del puzzle: nombre, total de piezas, ID (autogenerado o manual), URL de imagen                                                        |
| 3    | Por cada pieza (1 a N): descripción opcional, estado activo (default=Sí)                                                                        |
| 4    | Por cada pieza: ¿conecta con otra? Si sí: número de pieza destino y dirección (NORTE/SUR/ESTE/OESTE); la arista inversa se crea automáticamente |
| 5    | Selección de pieza de inicio del armado (número 1-N, Enter = primera activa)                                                                    |
| 6    | Resumen visual: piezas, faltantes, conexiones, punto de inicio                                                                                  |
| 7    | Confirmación final antes de cargar en Neo4j                                                                                                     |

### 6.2 Script de armado: armador.py

Aplicación interactiva de terminal que ejecuta el BFS y muestra el resultado paso a paso.

| Paso | Pregunta / acción                                                                                                   |
|------|---------------------------------------------------------------------------------------------------------------------|
| 1    | Verificación de credenciales y conexión a Neo4j Aura                                                                |
| 2    | Selección de puzzle (lista numerada de todos los puzzles disponibles)                                               |
| 3    | ¿Hay piezas faltantes? Si sí: ingresar IDs separados por coma (P_1, P_5, ...); se marcan como activa=false en la BD |

|   |                                                                |
|---|----------------------------------------------------------------|
| 4 | ¿Desde qué pieza iniciar? (Enter = usa PRIMER_PASO del puzzle) |
| 5 | Ejecución del BFS con salida paso a paso en terminal           |

### 6.3 Credenciales y configuración

Las credenciales de Neo4j Aura se almacenan en un archivo .env (excluido de control de versiones). El archivo requirements.txt lista las dependencias necesarias.

```
# .env
NEO4J_URI=neo4j+s://<instancia>.databases.neo4j.io
NEO4J_USERNAME=<usuario>
NEO4J_PASSWORD=<contraseña>
NEO4J_DATABASE=<base_de_datos>

# requirements.txt
neo4j>=5.0.0
python-dotenv>=1.0.0
```

## 7. Consultas Cypher de referencia

### 7.1 Creación del esquema

#### Crear el rompecabezas raíz

```
CREATE (:Puzzle {
  puzzle_id: 'puzzle_abc12345',
  nombre: 'Mi Rompecabezas',
  total_piezas: 9,
  dimensiones: '9 piezas asimétrico',
  imagen_url: ''
});
```

#### Crear piezas

```
CREATE (:Pieza {pieza_id: 'P_1', descripcion: 'Pieza esquina superior
izquierda', activa: true});
CREATE (:Pieza {pieza_id: 'P_2', descripcion: 'Pieza borde superior',
activa: true});
CREATE (:Pieza {pieza_id: 'P_5', descripcion: 'Pieza central', activa:
false}); // faltante
```

#### Crear relación de adyacencia CONECTA (bidireccional)

```
MATCH (a:Pieza {pieza_id: 'P_1'}), (b:Pieza {pieza_id: 'P_2'})
CREATE (a)-[:CONECTA {direccion: 'ESTE', compatible: true, fuerza_encaje:
1.0}]->(b)
CREATE (b)-[:CONECTA {direccion: 'OESTE', compatible: true, fuerza_encaje:
1.0}]->(a);
```

### Vincular inicio al nodo raíz (PRIMER\_PASO)

```
MATCH (pz:Puzzle {puzzle_id: 'puzzle_abc12345'}), (p:Pieza {pieza_id: 'P_1'})
CREATE (pz)-[:PRIMER_PASO]->(p);
```

## 7.2 Consultas de operación

### ¿Por dónde empezar el armado?

```
MATCH (pz:Puzzle {puzzle_id: $puzzle_id})-[:PRIMER_PASO]->(p:Pieza)
WHERE p.activa = true
RETURN p.pieza_id AS inicio
LIMIT 1;
```

### Obtener todos los vecinos de una pieza

```
MATCH (p:Pieza {pieza_id: $pieza_id})-[r:CONECTA]->(v:Pieza)
RETURN r.direccion AS direccion, v.pieza_id AS vecino, v.activa AS activa
ORDER BY r.direccion;
```

### Marcar una pieza como faltante

```
MATCH (p:Pieza {pieza_id: $pieza_id})
SET p.activa = false
RETURN p.pieza_id AS id;
```

### Recolocar una pieza faltante

```
MATCH (p:Pieza {pieza_id: $pieza_id})
SET p.activa = true
RETURN p.pieza_id AS id;
```

### Estado general del rompecabezas

```
MATCH (pz:Puzzle {puzzle_id: $puzzle_id})
MATCH (p:Pieza)
RETURN
    pz.total_piezas AS total,
    count(CASE WHEN p.activa = true THEN 1 END) AS ensambladas,
    count(CASE WHEN p.activa = false THEN 1 END) AS faltantes;
```

## 8. Índices y restricciones

Restricciones de unicidad que garantizan integridad referencial:

```
CREATE CONSTRAINT puzzle_id_unique IF NOT EXISTS
FOR (pz:Puzzle) REQUIRE pz.puzzle_id IS UNIQUE;

CREATE CONSTRAINT pieza_id_unique IF NOT EXISTS
FOR (p:Pieza) REQUIRE p.pieza_id IS UNIQUE;
```

Índice para acelerar el filtrado de piezas por estado activo:

```
CREATE INDEX pieza_activa_idx IF NOT EXISTS
FOR (p:Pieza) ON (p.activa);
```



## 9. Ejemplo completo — Rompecabezas 3×3 (9 piezas)

Script Cypher completo para un rompecabezas de 3×3. Las piezas se numeran en orden de lectura (izquierda a derecha, fila por fila):

```
P_1  P_2  P_3
P_4  P_5  P_6
P_7  P_8  P_9
```

### Crear nodo raíz

```
CREATE (:Puzzle {
  puzzle_id: 'puzzle_3x3',
  nombre: 'Mini 3x3',
  total_piezas: 9,
  dimensiones: '9 piezas asimétrico',
  imagen_url: ''
});
```

### Crear las 9 piezas

```
CREATE (:Pieza {pieza_id:'P_1', descripcion:'Esquina sup-izq',
activa:true});
CREATE (:Pieza {pieza_id:'P_2', descripcion:'Borde superior',
activa:true});
CREATE (:Pieza {pieza_id:'P_3', descripcion:'Esquina sup-der',
activa:true});
CREATE (:Pieza {pieza_id:'P_4', descripcion:'Borde izquierdo',
activa:true});
CREATE (:Pieza {pieza_id:'P_5', descripcion:'Centro',
activa:true});
CREATE (:Pieza {pieza_id:'P_6', descripcion:'Borde derecho',
activa:true});
CREATE (:Pieza {pieza_id:'P_7', descripcion:'Esquina inf-izq',
activa:true});
CREATE (:Pieza {pieza_id:'P_8', descripcion:'Borde inferior',
activa:true});
CREATE (:Pieza {pieza_id:'P_9', descripcion:'Esquina inf-der',
activa:true});
```

### Definir punto de inicio

```
MATCH (pz:Puzzle {puzzle_id:'puzzle_3x3'}), (p:Pieza {pieza_id:'P_1'})
CREATE (pz)-[:PRIMER_PASO]->(p);
```

### Relaciones horizontales (fila 1)

```
MATCH (a:Pieza {pieza_id:'P_1'}),(b:Pieza {pieza_id:'P_2'})
CREATE (a)-[:CONECTA {direccion:'ESTE',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'OESTE',compatible:true,fuerza_encaje:1.0}]->(a);

MATCH (a:Pieza {pieza_id:'P_2'}),(b:Pieza {pieza_id:'P_3'})
CREATE (a)-[:CONECTA {direccion:'ESTE',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'OESTE',compatible:true,fuerza_encaje:1.0}]->(a);
```

### Relaciones horizontales (fila 2)

```
MATCH (a:Pieza {pieza_id:'P_4'}),(b:Pieza {pieza_id:'P_5'})
CREATE (a)-[:CONECTA {direccion:'ESTE',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'OESTE',compatible:true,fuerza_encaje:1.0}]->(a);

MATCH (a:Pieza {pieza_id:'P_5'}),(b:Pieza {pieza_id:'P_6'})
CREATE (a)-[:CONECTA {direccion:'ESTE',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'OESTE',compatible:true,fuerza_encaje:1.0}]->(a);
```

### Relaciones horizontales (fila 3)

```
MATCH (a:Pieza {pieza_id:'P_7'}),(b:Pieza {pieza_id:'P_8'})
CREATE (a)-[:CONECTA {direccion:'ESTE',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'OESTE',compatible:true,fuerza_encaje:1.0}]->(a);

MATCH (a:Pieza {pieza_id:'P_8'}),(b:Pieza {pieza_id:'P_9'})
CREATE (a)-[:CONECTA {direccion:'ESTE',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'OESTE',compatible:true,fuerza_encaje:1.0}]->(a);
```

### Relaciones verticales (columnas 1, 2, 3)

```
// Columna 1
MATCH (a:Pieza {pieza_id:'P_1'}),(b:Pieza {pieza_id:'P_4'})
CREATE (a)-[:CONECTA {direccion:'SUR',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'NORTE',compatible:true,fuerza_encaje:1.0}]->(a);
MATCH (a:Pieza {pieza_id:'P_4'}),(b:Pieza {pieza_id:'P_7'})
CREATE (a)-[:CONECTA {direccion:'SUR',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'NORTE',compatible:true,fuerza_encaje:1.0}]->(a);

// Columna 2
MATCH (a:Pieza {pieza_id:'P_2'}),(b:Pieza {pieza_id:'P_5'})
CREATE (a)-[:CONECTA {direccion:'SUR',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'NORTE',compatible:true,fuerza_encaje:1.0}]->(a);
MATCH (a:Pieza {pieza_id:'P_5'}),(b:Pieza {pieza_id:'P_8'})
CREATE (a)-[:CONECTA {direccion:'SUR',
compatible:true,fuerza_encaje:1.0}]->(b),
      (b)-[:CONECTA
{direccion:'NORTE',compatible:true,fuerza_encaje:1.0}]->(a);

// Columna 3
MATCH (a:Pieza {pieza_id:'P_3'}),(b:Pieza {pieza_id:'P_6'})
CREATE (a)-[:CONECTA {direccion:'SUR',
```

```
compatible:true,fuerza_encaje:1.0}}->(b),
    (b)-[:CONECTA
{direccion:'NORTE',compatible:true,fuerza_encaje:1.0}}->(a);
MATCH (a:Pieza {pieza_id:'P_6'}),(b:Pieza {pieza_id:'P_9'})
CREATE (a)-[:CONECTA {direccion:'SUR',
compatible:true,fuerza_encaje:1.0}}->(b),
    (b)-[:CONECTA
{direccion:'NORTE',compatible:true,fuerza_encaje:1.0}}->(a);
```